

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2021 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2021	2
SUBIECTUL I (60 de puncte)	2
1. Tablouri unidimensionale și vectori	2
2. Structura de directoare și fișiere	2
3. Programare: Cifre pare și impare	2
4. Baze de date: Zboruri companie	4
SUBIECTUL al II-lea - Completare pentru Model 2021 (30 de puncte)	5
1. Activitate didactică pentru pseudocod și structuri repetitive	5
2. Itemi de tip eseu	5
SUBIECTUL al II-lea - Completare pentru Varianta 3 2021 (30 de puncte)	5
1. Strategie didactică pentru secvența A: grafuri neorientate	5
2. Itemi cu alegere multiplă	5
II. Rezolvare Definitivat Varianta 3 (14 iulie 2021)	6
SUBIECTUL I (60 de puncte)	6
1. Algoritmul de sortare prin metoda Bulelor (Bubble Sort)	6
a) Descriere în limbaj natural și exemplificare pe 7 numere	6
b) Complexitatea algoritmului	6
c) Problemă completă (Bubble Sort)	6
2. Organizarea logică a datelor în cadrul unui sistem de operare	7
3. Programare: Cifre din mulțimea $\{0, 1, 2\}$	7
4. Baze de date: Companie Aviatică	9

I. Rezolvare Definitivat Model 2021

SUBIECTUL I (60 de puncte)

1. Tablouri unidimensionale și vectori

- **Declarare:** Alocare statică `int V[100]` / `V: array[1..100] of integer` și alocare dinamică.
- **Elemente:** Relația dintre indici și deplasare în vector.
- **Problemă (Determinarea valorii minime):**

► **C++:**

```
#include <iostream>
using namespace std;
int main() {
    int n, V[100];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> V[i];
    int minim = V[0];
    for (int i = 1; i < n; ++i) if (V[i] < minim) minim = V[i];
    cout << minim << endl;
    return 0;
}
```

► **Pascal:**

```
program MinimVector;
var
    n, i, minim: integer;
    V: array[1..100] of integer;
begin
    read(n);
    for i := 1 to n do read(V[i]);
    minim := V[1];
    for i := 2 to n do if V[i] < minim then minim := V[i];
    writeln(minim);
end.
```

2. Structura de directoare și fișiere

- **Sistem de operare:** Gestionează fișierele prin intermediul tabelor de alocare a fișierelor (ex. NTFS, ext4).
- **Directoare și căi:** Calea absolută (de la rădăcină) și calea relativă (față de directorul curent).
- **Operații:** Redenumire, ștergere, creare, copiere.

3. Programare: Cifre pare și impare

Soluție C++:

```
#include <iostream>
#include <fstream>
using namespace std;

int sumaCifre(long long n) {
    int s = 0;
    while (n > 0) {
        s += n % 10;
    }
}
```

```

        n /= 10;
    }
    return s;
}

int main() {
    ifstream fin("def2021.in");
    long long x;
    int count_pare = 0, count_impair = 0;
    while (fin >> x) {
        int sc = sumaCifre(x);
        if (sc % 2 == 0) count_pare++;
        else count_impair++;
    }
    fin.close();

    ofstream fout("def2021.out");
    fout << count_pare << " " << count_impair << "\n";
    fout.close();

    return 0;
}

```

Soluție Pascal:

```

program ParitateCifre;
var
    fin, fout: text;
    x: int64;
    count_pare, count_impair: integer;

function sumaCifre(n: int64): integer;
var s: integer;
begin
    s := 0;
    while n > 0 do
    begin
        s := s + (n mod 10);
        n := n div 10;
    end;
    sumaCifre := s;
end;

begin
    assign(fin, 'def2021.in');
    reset(fin);
    count_pare := 0;
    count_impair := 0;
    while not eof(fin) do
    begin
        read(fin, x);
        if sumaCifre(x) mod 2 = 0 then
            count_pare := count_pare + 1
        else
            count_impair := count_impair + 1;
        end;
    end;
    close(fin);
end;

```

```

assign(fout, 'def2021.out');
rewrite(fout);
writeln(fout, count_pare, ' ', count_impere);
close(fout);
end.

```

4. Baze de date: Zboruri companie

Model conceptual:

- CLIENT: id_client (PK), nume, prenume, adresa.
- ZBOR: id_zbor (PK), aeroport_sursa, aeroport_destinatie, data_ora_plecure.
- BILET: id_bilet (PK), id_client (FK), id_zbor (FK), mod_plata.

Model fizic:

```

CREATE TABLE CLIENT (
    id_client INT PRIMARY KEY,
    nume VARCHAR(50) NOT NULL,
    prenume VARCHAR(50) NOT NULL,
    adresa VARCHAR(150)
);

CREATE TABLE ZBOR (
    id_zbor INT PRIMARY KEY,
    aeroport_sursa VARCHAR(80) NOT NULL,
    aeroport_destinatie VARCHAR(80) NOT NULL,
    data_ora_plecure DATETIME NOT NULL
);

CREATE TABLE BILET (
    id_bilet INT PRIMARY KEY,
    id_client INT NOT NULL,
    id_zbor INT NOT NULL,
    mod_plata VARCHAR(30) NOT NULL,
    FOREIGN KEY (id_client) REFERENCES CLIENT(id_client),
    FOREIGN KEY (id_zbor) REFERENCES ZBOR(id_zbor)
);

```

SQL pentru clienții care au plătit numerar:

```

SELECT DISTINCT c.prenume, c.nume
FROM CLIENT c
JOIN BILET b ON c.id_client = b.id_client
WHERE b.mod_plata = 'numerar';

```

- **Relații și normalizare:** CLIENT 1:M BILET, ZBOR 1:M BILET; tabela BILET materializează achizițiile și evită repetarea datelor clientului/zborului. Modelul respectă 1NF prin câmpuri atomice, 2NF prin dependența completă a atributelor de cheia fiecărei tabele și 3NF prin separarea datelor clientului, zborului și plății. Cheile străine id_client, id_zbor trebuie validate, iar mod_plata se recomandă controlat prin domeniu de valori.
-

SUBIECTUL al II-lea - Completare pentru Model 2021 (30 de puncte)

1. Activitate didactică pentru pseudocod și structuri repetitive

Momente: organizare; actualizarea cunoștințelor; predare și dirijarea învățării; fixare/evaluare.

Activități: analizarea unei probleme în limbaj natural; scrierea pseudocodului cu structură repetitivă pentru suma cifrelor.

Scenariu: Profesorul propune problema, elevii identifică operațiile `mod` și `div`, apoi formulează pseudocodul. Profesorul verifică respectarea programării structurate: inițializare, condiție, actualizare, rezultat.

2. Itemi de tip eseu

Caracteristici: răspuns elaborat, criterii analitice, evaluarea argumentării. **Regulă:** repere clare. **Categorii:** eseu structurat și eseu liber.

- **Item A:** Eseu structurat despre importanța pseudocodului în proiectarea algoritmilor. **Criterii:** concepte 5p, exemple 5p, coerență 5p.
- **Item B:** Eseu despre avantajele rețelelor LAN/WAN/Internet și partajarea resurselor. **Criterii:** definiții 5p, avantaje 5p, exemple 5p.

SUBIECTUL al II-lea - Completare pentru Varianta 3 2021 (30 de puncte)

1. Strategie didactică pentru secvența A: grafuri neorientate

Mijloc: aplicație de desenare a grafurilor sau tablă interactivă. **Caracteristică:** permite reprezentarea vizuală a nodurilor și muchiilor. **Avantaj:** elevii transpun mai ușor enunțul în limbaj de grafuri.

Metodă: învățarea prin descoperire. **Formă:** grupe mici. **Activitate:** identificarea lanțurilor și a componentelor conexe într-un graf dat.

Scenariu: Profesorul prezintă o rețea de localități, elevii construiesc graful, marchează noduri/muchii, identifică lanțuri elementare și discută conectivitatea.

2. Itemi cu alegere multiplă

Caracteristici: au enunț clar, alternative plauzibile, un singur răspuns corect. **Reguli:** distractori omogeni, fără indicii gramaticale, evitarea răspunsurilor ambigue.

- **Item A:** Într-un graf neorientat, un lanț elementar este: A. un lanț cu muchii orientate B. un lanț cu noduri distincte C. un graf complet D. o matrice. **Răspuns:** B.
- **Item B:** Care este dispozitiv de intrare? A. Monitor B. Imprimantă C. Tastatură D. Boxe. **Răspuns:** C.

II. Rezolvare Definitivat Varianta 3 (14 iulie 2021)

SUBIECTUL I (60 de puncte)

1. Algoritmul de sortare prin metoda Bulelor (Bubble Sort)

a) Descriere în limbaj natural și exemplificare pe 7 numere

- **Descriere:** Algoritmul parcurge tabloul în mod repetat și compară elementele adiacente. Dacă două elemente consecutive nu sunt în ordinea corectă (crescătoare), acestea sunt interschimbate. Parcurgerile se repetă până când nu mai are loc nicio interschimbare într-o trecere completă.
- **Exemplificare pentru tabloul:** [25, 14, 30, 8, 12, 5, 20] ($N = 7$)
 - **Trecerea 1:**
 - $25 > 14 \rightarrow$ swap $\rightarrow$ [14, 25, 30, 8, 12, 5, 20]
 - $25 < 30 \rightarrow$ no swap
 - $30 > 8 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 30, 12, 5, 20]
 - $30 > 12 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 30, 5, 20]
 - $30 > 5 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 5, 30, 20]
 - $30 > 20 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 5, 20, 30] (30 a ajuns pe poziția corectă).
 - **Trecerea 2:** Parcurgem până la penultimul element:
 - Interschimbările succesive aduc elementul 25 pe penultima poziție: [14, 8, 12, 5, 20, 25, 30].
 - **Trecerile 3, 4, 5, 6** continuă în același mod până când vectorul devine complet sortat: [5, 8, 12, 14, 20, 25, 30].

b) Complexitatea algoritmului

- **Cazul cel mai nefavorabil (Worst case):** Vectorul este sortat descrescător. Se efectuează $O(N^2)$ comparații și interschimbări.
- **Cazul cel mai favorabil (Best case):** Vectorul este deja sortat. Cu o optimizare ce folosește un fanion (flag) de verificare a interschimbărilor, algoritmul se oprește după prima trecere: $O(N)$.
- **Cazul mediu (Average case):** Complexitatea timpului este de ordinul $O(N^2)$.

c) Problemă completă (Bubble Sort)

• Cod C++:

```
#include <iostream>
using namespace std;
void bubbleSort(int A[], int n) {
    bool sortat;
    do {
        sortat = true;
        for (int i = 0; i < n - 1; ++i) {
            if (A[i] > A[i + 1]) {
                swap(A[i], A[i + 1]);
                sortat = false;
            }
        }
    } while (!sortat);
}
int main() {
```

```
int A[] = {25, 14, 30, 8, 12, 5, 20};
bubbleSort(A, 7);
for (int x : A) cout << x << " ";
return 0;
}
```

2. Organizarea logică a datelor în cadrul unui sistem de operare

• Structura sistemului de fișiere:

1. **Partiție:** O diviziune logică a unui hard disk fizic tratată de SO ca un volum independent.
2. **Director:** O structură logică de tip container ce conține referințe către fișiere și alte directoare.
3. **Fișier:** O colecție organizată de informații stocată pe un mediu extern.

• Operații:

- **Formatare:** Pregătirea unei partiții pentru a fi folosită de SO prin crearea structurii sistemului de fișiere (ex. NTFS, FAT32).
- **Copiere, Mutare, Redenumire, Ștergere** de directoare/fișiere.

3. Programare: Cifre din mulțimea {0, 1, 2}

- **Descriere:** Subprogramul `cifDistMax` numără cifrele distincte și determină cifra maximă a lui n . Programul principal generează recursiv toate numerele formate exclusiv din cifrele {0, 1, 2}, le ordonează și utilizează căutarea binară pentru a determina cel mai mic număr din intervalul $[x, y]$. După găsirea candidatului, se apelează `cifDistMax` pentru validarea proprietății cerute: cifra maximă trebuie să fie cel mult 2.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <queue>
using namespace std;

void cifDistMax(long long n, int &nrd, int &cma) {
    bool ap[10] = {false};
    nrd = 0; cma = 0;
    long long temp = n;
    if (temp == 0) { nrd = 1; cma = 0; return; }
    while (temp > 0) {
        int cif = temp % 10;
        if (!ap[cif]) { ap[cif] = true; nrd++; }
        if (cif > cma) cma = cif;
        temp /= 10;
    }
}

vector<long long> vals;
void genereaza() {
    vals.push_back(0);
    queue<long long> Q;
    Q.push(1); Q.push(2);
}
```

```

while (!Q.empty()) {
    long long u = Q.front(); Q.pop();
    if (u > 1000000000LL) continue;
    vals.push_back(u);
    Q.push(u * 10 + 0);
    Q.push(u * 10 + 1);
    Q.push(u * 10 + 2);
}
sort(vals.begin(), vals.end());
}

int main() {
    genereaza();
    long long x, y;
    if (!(cin >> x >> y)) return 0;

    ofstream fout("def.out");
    auto it = lower_bound(vals.begin(), vals.end(), x);
    int nrd, cma;
    if (it != vals.end()) cifDistMax(*it, nrd, cma);
    if (it != vals.end() && *it <= y && cma <= 2) {
        fout << *it << "\n";
    } else {
        fout << "nu exista\n";
    }
    fout.close();
    return 0;
}

```

Soluție Pascal:

```

program NumereTernare;
var
    x, y: int64;
    vals: array[1..60000] of int64;
    count, i, st, dr, mid, pos, nrd, cma: integer;
    fout: text;

procedure gen(val: int64);
begin
    if val > 1000000000 then exit;
    count := count + 1;
    vals[count] := val;
    gen(val * 10 + 0);
    gen(val * 10 + 1);
    gen(val * 10 + 2);
end;

procedure QuickSort(l, r: integer);
var
    i, j: integer;
    pivot, tmp: int64;
begin
    i := l; j := r;
    pivot := vals[(l + r) div 2];
    repeat
        while vals[i] < pivot do i := i + 1;

```



```

while vals[j] > pivot do j := j - 1;
if i <= j then
begin
    tmp := vals[i]; vals[i] := vals[j]; vals[j] := tmp;
    i := i + 1; j := j - 1;
end;
until i > j;
if l < j then QuickSort(l, j);
if i < r then QuickSort(i, r);
end;

begin
    readln(x, y);
    count := 1;
    vals[1] := 0;
    gen(1);
    gen(2);
    QuickSort(1, count);

    st := 1; dr := count; pos := -1;
    while st <= dr do
    begin
        mid := (st + dr) div 2;
        if vals[mid] >= x then
        begin
            pos := mid;
            dr := mid - 1;
        end
        else st := mid + 1;
    end;

    assign(fout, 'def.out');
    rewrite(fout);
    if pos <> -1 then
    begin
        cifDistMax(vals[pos], nrd, cma);
        if (vals[pos] <= y) and (cma <= 2) then
            writeln(fout, vals[pos])
        else
            writeln(fout, 'nu exista');
        end
    else
        writeln(fout, 'nu exista');
    close(fout);
end.

```

4. Baze de date: Companie Aviatică

Model conceptual:

- CLIENT: id_client (PK), nume, prenume, adresa.
- ZBOR: id_zbor (PK), aeroport_sursa, aeroport_destinatie, data_ora_plecare, data_ora_sosire.
- BILET: id_bilet (PK), id_client (FK), id_zbor (FK), mod_plata, data_achizitie.

Model fizic:

```
CREATE TABLE CLIENT (
    id_client INT PRIMARY KEY,
    nume VARCHAR(50) NOT NULL,
    prenume VARCHAR(50) NOT NULL,
    adresa VARCHAR(150)
);

CREATE TABLE ZBOR (
    id_zbor INT PRIMARY KEY,
    aeroport_sursa VARCHAR(80) NOT NULL,
    aeroport_destinatie VARCHAR(80) NOT NULL,
    data_ora_plecure DATETIME NOT NULL,
    data_ora_sosire DATETIME NOT NULL,
    CHECK (data_ora_sosire > data_ora_plecure)
);

CREATE TABLE BILET (
    id_bilet INT PRIMARY KEY,
    id_client INT NOT NULL,
    id_zbor INT NOT NULL,
    mod_plata VARCHAR(30) NOT NULL,
    data_achizitie DATE NOT NULL,
    FOREIGN KEY (id_client) REFERENCES CLIENT(id_client),
    FOREIGN KEY (id_zbor) REFERENCES ZBOR(id_zbor)
);
```

SQL pentru clienții distincți care au plătit numerar:

```
SELECT DISTINCT c.prenume, c.nume
FROM CLIENT c
JOIN BILET b ON c.id_client = b.id_client
WHERE b.mod_plata = 'numerar';
```

- **Relații și normalizare:** CLIENT 1:M BILET, ZBOR 1:M BILET; tabela BILET păstrează data achiziției și modul de plată pentru fiecare cumpărare. Atributele sunt atomice (1NF), depind de cheia propriei tabele (2NF), iar datele clientului și ale zborului nu sunt duplicate în BILET (3NF). Restricții utile: data_ora_sosire > data_ora_plecure, mod_plata din listă controlată, chei străine obligatorii.